



자바응용SW(앱)개발자양성

어댑터 뷰

백제직업전문학교

강사 : 김영준

스크롤뷰의 한계

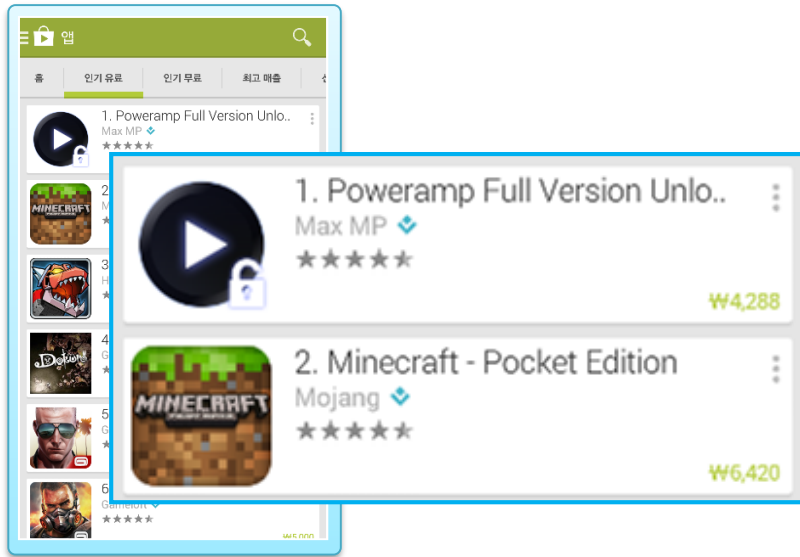
- 스크롤뷰는 자신의 영역을 초과하여 배치된 자식 뷰를 스크롤하며 볼 수 있게 한다.
- 하지만 스크롤 전에는 보이지 않을 자식 뷰까지 미리 생성하고 그려두기 때문에 자식 뷰가 많을 수록 메모리 사용량이 늘어난다.



- 플리킹하면 아래에 무수히 많은 뷰가 보여진다.
- 만일 해당 앱에서 무수히 많은 뷰를 LinearLayout으로 수직 배치하고, 스크롤뷰를 이용해서 이동한다면 앱은 메모리 부족으로 바로 죽어버릴 것이다.
- 이러한 문제를 극복하고자 안드로이드는 어댑터뷰를 제공한다.

스크롤뷰와 어댑터뷰

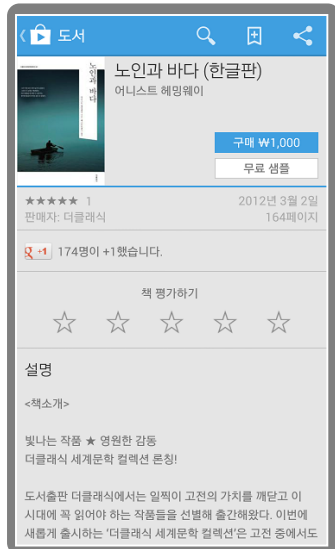
■ 어댑터뷰는 어떤환경에서 사용되는지 살펴보자.



■ 이 화면은 콘텐츠의 내용만 다른 **동일한 구조의 레이아웃**들이 수직으로 그룹을 이루고 있다.

■ 바로 이 경우 **어댑터뷰**를 사용한다.

■ 어댑터뷰는 **현재 화면에 보이지 않는 자식 뷰**를 미리 생성하지 않고 스크롤되어 **보여야 할 때 비로소** 생성된다.



■ 콘텐츠 내용이 모두 다르며,
동일한 형태의 레이아웃 그룹을 형성하고 있지 않다.

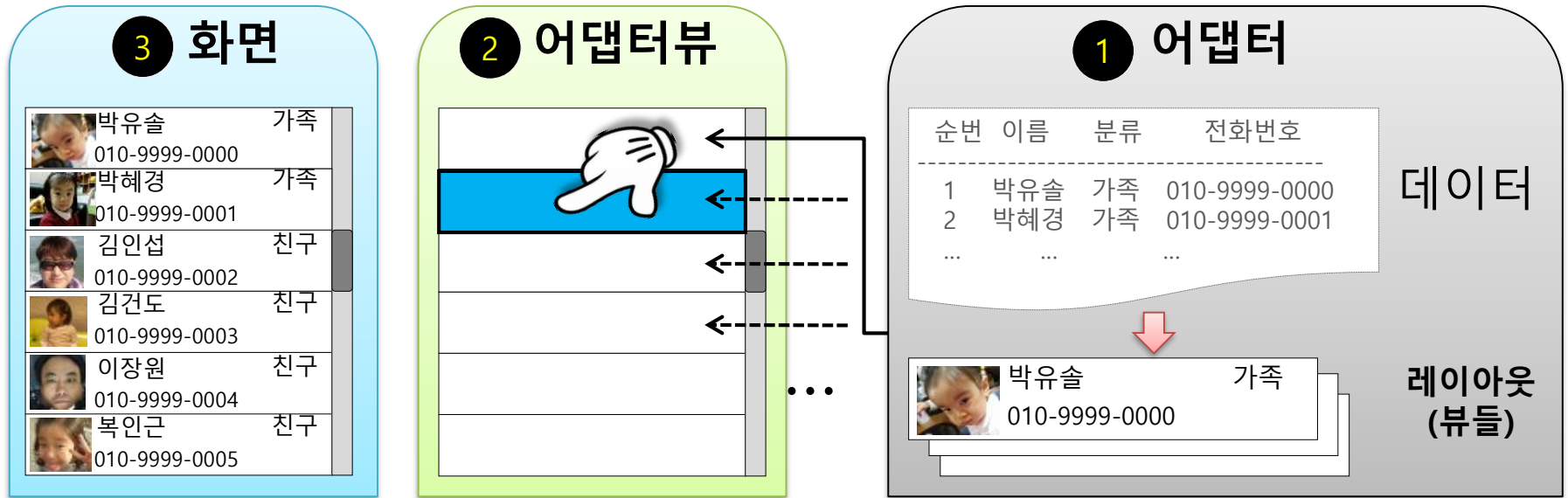
■ 이렇게 뷰마다 배치가 다양한 레이아웃은 **스크롤뷰**를 사용한다.

■ 스크롤뷰는 보이지 않는 영역의 뷰까지 모두 생성하고 그리기 때문에 배치할 뷰의 수가 제한적이다.

하지만 어댑터뷰와 같이 유사한 성격의 콘텐츠를 리스트로 배치하는 구조가 아니라면 한 화면에 그렇게 많은 뷰를 사용할 일이 거의 없겠다.

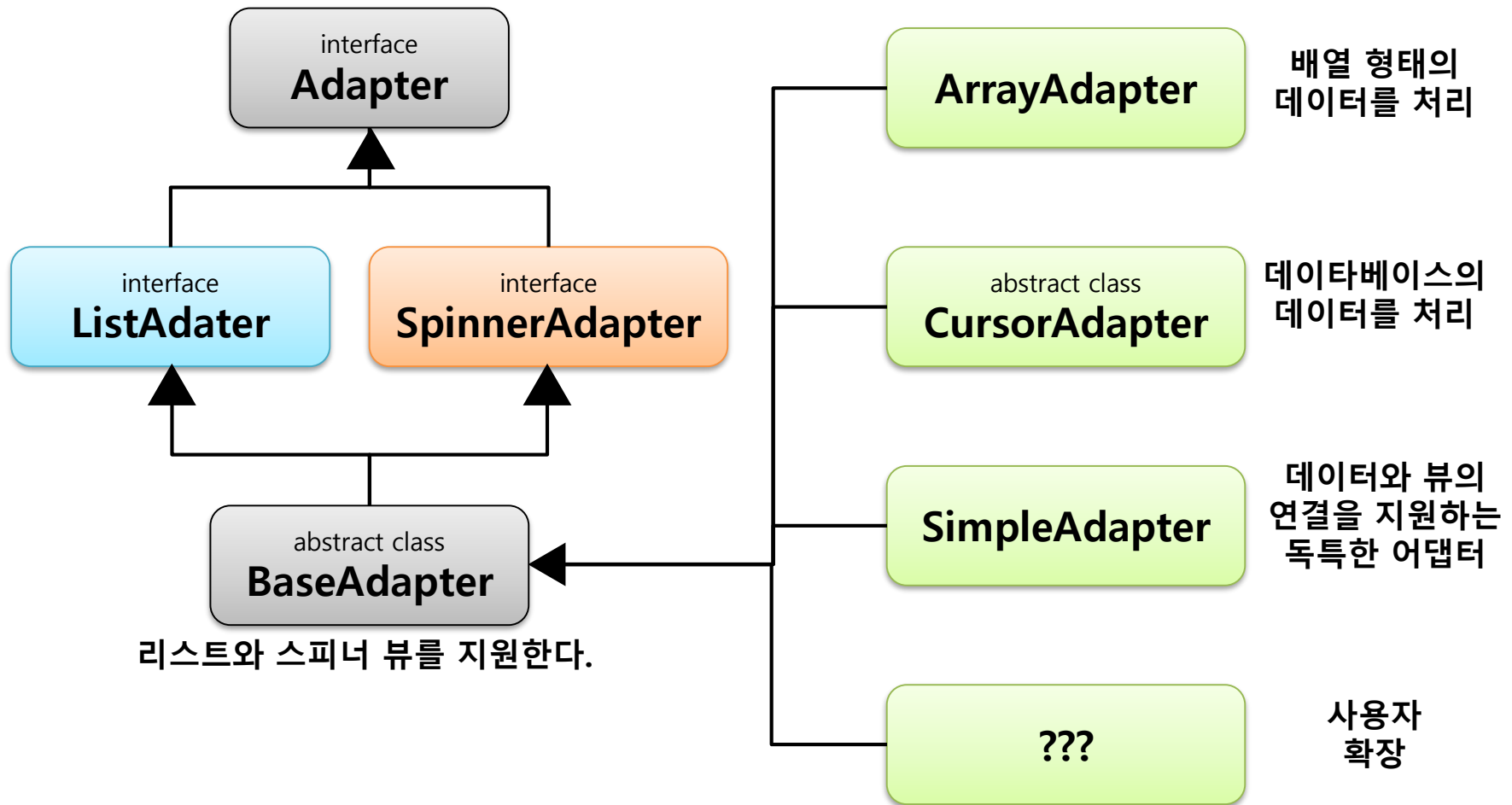
- 어댑터뷰AdapterView는 기존의 어떤 뷰그룹보다 사용법이 어렵고 복잡하다.
- 그렇다고 해서 많이 사용되지 않는 것도 아니다. 일반적으로 앱을 구현하다 보면 반드시 하나 이상의 어댑터뷰를 사용해야 하는 경우가 대부분이다. 즉 피해갈 수 없는 필수 뷰그룹이라는 의미다.
- 하지만 구조와 원리를 알고 나면 어렵기는 커녕 오히려 '동일한 성격의 많은 데이터를 이렇게 쉽게 어댑터 뷰로 보여줄 수 있구나'라는 감탄이 들 정도일 것이다.

어댑터뷰 - 어댑터뷰와 어댑터



- 어댑터뷰와 어댑터는 철저하게 분리되어 독립적으로 동작한다.
- 여기서 어댑터뷰는 배치되는 모습에 따라 다양하고,
어댑터 역시 관리되는 데이터와 어댑터 뷰에게 제공할 뷰 형태에 따라 그 종류가 다양하다.
- 이렇게 서로 다양한 어댑터뷰와 어댑터는 구현하는 앱에 따라 조합하여 사용할 수 있는 구조다.

어댑터뷰 - 다양한 어댑터



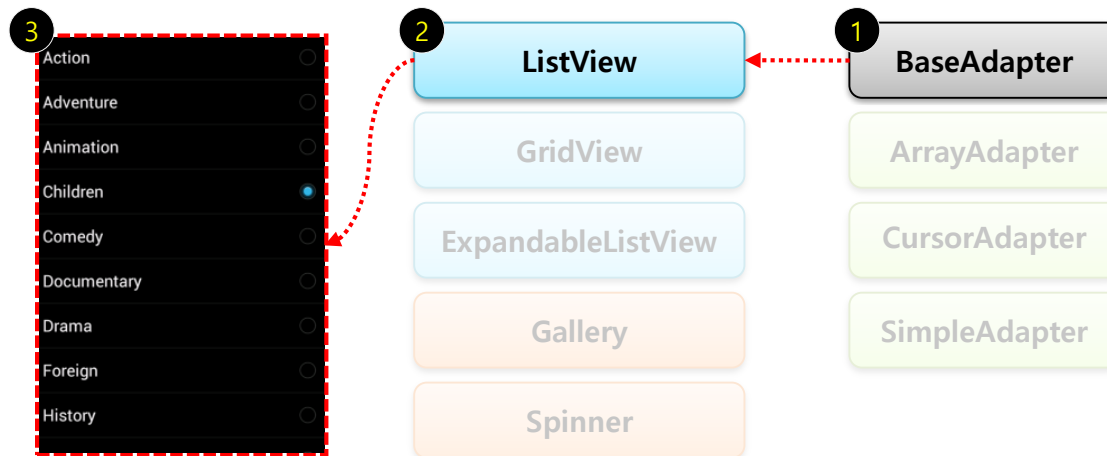
■ BaseAdapter는 리스트와 스피너 어댑터 모두를 상속받는다.

또한 대부분의 어댑터뷰가 사용하는 어댑터는 해당 클래스 혹은 파생된 클래스들을 사용한다.

그러므로 리스트와 스피너의 구분은 별 의미가 없는 것이다.

BaseAdapter를 이용한 리스트뷰

- 지금부터 어댑터뷰와 어댑터를 이해하기 위해서 리스트뷰를 실습해보자.
- 리스트뷰에서 사용할 어댑터는 **BaseAdapter**를 사용할 것이다.
그 이유는 BaseAdapter를 직접 구현해 봄으로써 파생된 기타 어댑터의 특징을 쉽게 이해할 수 있기 때문이다.
- 리스트뷰는 어댑터뷰 중 가장 많이 사용되는 뷰다. 따라서 **리스트뷰**를 사용한다.



BaseAdapter를 이용한 리스트뷰 - 데이터 만들기

■ 리스트를 구성하기 위해 가장 먼저 해야할 것은 바로 리스트로 보여줄 데이터를 준비하는 과정이다.

● 총 100개의 학생 데이터

이름	학번	학과
슈퍼성근0	950000	컴퓨터 공학0
슈퍼성근1	950001	컴퓨터 공학1
...	...	...
슈퍼성근100	9500099	컴퓨터 공학99



● 리스트로 표현

ListView	
슈퍼성근0 컴퓨터 공학0	950000
슈퍼성근1 컴퓨터 공학1	950001
슈퍼성근2	950002

src/Students.java

```
public class Student
{
    String mName      = ""; // 이름
    String mNumber    = ""; // 학번
    String mDepartment = ""; // 학과
}
```


BaseAdapter를 이용한 리스트뷰 - 데이터 만들기

src/MainActivity.java

```
public class MainActivity extends Activity
{
    ArrayList<Student> mData = null;

    @Override
    protected void onCreate( Bundle savedInstanceState )
    {
        super.onCreate( savedInstanceState );
        setContentView( R.layout.activity_main );

        // 어댑터에서 사용할 데이터 설정
        mData = new ArrayList<Student>();

        for( int i = 0 ; i < 100 ; i++ )
        {
            Student student = new Student();

            student.mName = "슈퍼성근" + i;
            student.mNumber = "95000" + i;
            student.mDepartment = "컴퓨터 공학" + i;

            mData.add( student );
        }
    }
}
```

- 예제에서는 데이터를 임의로 만들었지만 실제 앱에서는 대부분 원본 데이터를 DB나 파일, 네트워크를 통해 준비하면 된다.

BaseAdapter를 이용한 리스트뷰 - BaseAdapter 구현

- 다음은 준비된 학생 데이터를 BaseAdapter에 전달하고,
BaseAdapter는 전달받은 데이터를 이용해서 리스트뷰에게 전달할 뷰를 생성한다.

src/BaseAdapterEx.java

```
public class BaseAdapterEx extends BaseAdapter
```

```
{  
    Context          mContext = null;  
    ArrayList<Student> mData   = null;  
    LayoutInflater    mLayoutInflater = null;  
  
    public BaseAdapterEx( Context context, ArrayList<Student> data )  
    {  
        mContext      = context;  
        mData          = data;  
        mLayoutInflater = LayoutInflater.from(mContext);  
    }  
  
    public int getCount()  
    {  
        return mData.size();  
    }  
  
    public long getItemId( int position )  
    {  
        return position;  
    }  
  
    public Student getItem( int position )  
    {  
        return mData.get( position );  
    }  
  
    @Override  
    public View getView( int position,  
                        View convertView,  
                        ViewGroup parent )  
    {  
        return null;  
    }  
}
```

```
public Student getItem( int position )
```

```
{  
    return mData.get( position );  
}
```

```
@Override
```

```
public View getView( int position,  
                    View convertView,  
                    ViewGroup parent )
```

```
{  
    return null;  
}
```

```
}
```

BaseAdapter를 이용한 리스트뷰 - BaseAdapter 구현

■ getView는 리스트뷰의 아이템 하나에 들어갈 뷰를 전달하는 함수다.

res/layout/list_view_item_layout.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="wrap_content"
    android:layout_height="match_parent"
    android:padding="10dp">

    <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <TextView android:id="@+id/name_text"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="20dp"/>

        <TextView android:id="@+id/number_text"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textSize="20dp"
            android:gravity="right"/>

    </LinearLayout>

    <TextView android:id="@+id/department_text"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

BaseAdapter를 이용한 리스트뷰 - BaseAdapter 구현

■ getView 함수를 마저 구현한다.

src/BaseAdapterEx.java

```
public class BaseAdapterEx extends BaseAdapter
{
```

슈퍼성근0
컴퓨터 공학0

950000

```
    ...
    @Override
    public View getView( int position, View convertView, ViewGroup parent )
    {
        // 1. 리스트의 한 항목에 해당하는 레이아웃을 생성한다.
        View itemLayout = mLayoutInflater.inflate( R.layout.list_view_item_layout, null );

        TextView nameTv = (TextView) itemLayout.findViewById( R.id.name_text );
        TextView numberTv = (TextView) itemLayout.findViewById( R.id.number_text );
        TextView departmentTv = (TextView) itemLayout.findViewById( R.id.department_text );

        // 2. 이름, 학번, 학과 데이터를 참조하여 레이아웃을 갱신한다.
        nameTv.setText( mData.get( position ).mName );
        numberTv.setText( mData.get( position ).mNumber );
        departmentTv.setText( mData.get( position ).mDepartment );

        return itemLayout;
    }
}
```

■ getView 함수는 리스트뷰가 자신의 아이템을 그려야 할 때 호출한다.

BaseAdapter를 이용한 리스트뷰 -

BaseAdapter를 리스트뷰에 연결하기

src/MainActivity.java

```
public class MainActivity extends Activity
{
    ListView                    mListView = null;
    BaseAdapterEx              mAdapter = null;
    ArrayList<Student>         mData     = null;

    @Override
    protected void onCreate( Bundle savedInstanceState )
    {
        super.onCreate( savedInstanceState );
        setContentView( R.layout.activity_main );

        ...

        // 2. 어댑터를 생성하고 데이터 설정
        mAdapter = new BaseAdapterEx( this, mData );

        // 3. 리스트뷰에 어댑터 설정
        mListView = (ListView) findViewById( R.id.list_view );
        mListView.setAdapter( mAdapter );
    }
}
```

BaseAdapter를 이용한 리스트뷰 -

BaseAdapter를 리스트뷰에 연결하기

ListView	
슈퍼성근0 컴퓨터 공학0	950000
슈퍼성근1 컴퓨터 공학1	950001
슈퍼성근2 컴퓨터 공학2	950002
슈퍼성근3 컴퓨터 공학3	950003
슈퍼성근4 컴퓨터 공학4	950004
슈퍼성근5 컴퓨터 공학5	950005
슈퍼성근6 컴퓨터 공학6	950006

ListView	
컴퓨터 공학58	
슈퍼성근59 컴퓨터 공학59	9500059
슈퍼성근60 컴퓨터 공학60	9500060
슈퍼성근61 컴퓨터 공학61	9500061
슈퍼성근62 컴퓨터 공학62	9500062
슈퍼성근63 컴퓨터 공학63	9500063
슈퍼성근64 컴퓨터 공학64	9500064

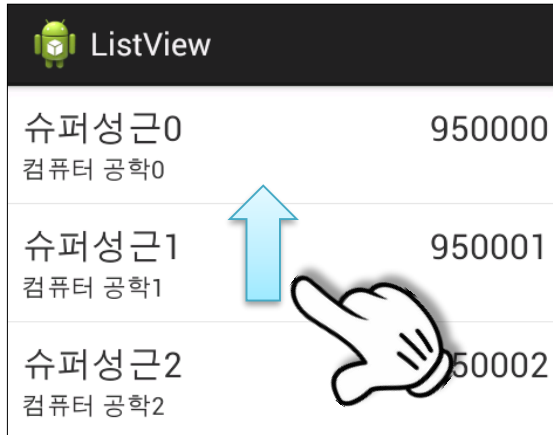
ListView	
슈퍼성근93 컴퓨터 공학93	9500093
슈퍼성근94 컴퓨터 공학94	9500094
슈퍼성근95 컴퓨터 공학95	9500095
슈퍼성근96 컴퓨터 공학96	9500096
슈퍼성근97 컴퓨터 공학97	9500097
슈퍼성근98 컴퓨터 공학98	9500098
슈퍼성근99 컴퓨터 공학99	9500099

■ 혹시 여러분은 리스트를 사용한 여러 앱 중, 빠르게 아래위로 이동했을 때 뚝뚝 끊기는 등의 부드럽지 않은 앱은 없었는가?

■ 예제 앱도 에뮬레이터에서 동작시키면 리스트가 부드럽게 움직이지 않을 것이다.
이는 어댑터의 최적화를 고려하지 않았기 때문이다.

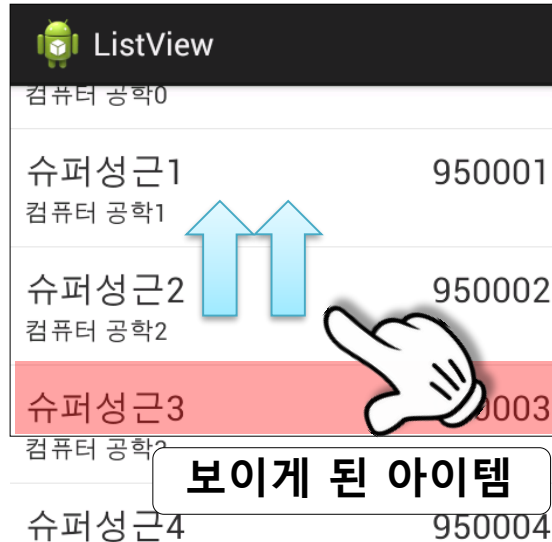
BaseAdapter를 이용한 리스트뷰 - 어댑터의 최적화

■ 어댑터의 최적화는 선택사항이 아니라 반드시 고려되어야 하는 필수사항이다.



ListView	
슈퍼성근0 컴퓨터 공학0	950000
슈퍼성근1 컴퓨터 공학1	950001
슈퍼성근2 컴퓨터 공학2	950002

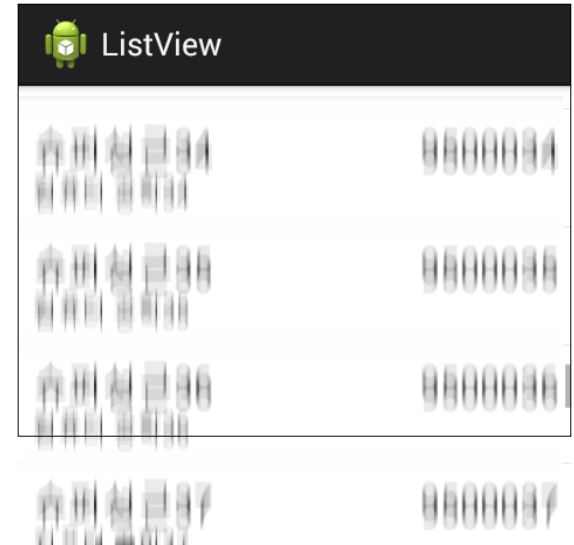
보이지 않는 아이템



ListView	
컴퓨터 공학0	
슈퍼성근1 컴퓨터 공학1	950001
슈퍼성근2 컴퓨터 공학2	950002
슈퍼성근3 컴퓨터 공학3	950003
슈퍼성근4 컴퓨터 공학4	950004

보이게 된 아이템

■ 바로 이때 Adapter의
getView 함수가 호출되고
보이게 될 **아이템을**
생성하게 된다.



ListView	
슈퍼성근0 컴퓨터 공학0	950000
슈퍼성근1 컴퓨터 공학1	950001
슈퍼성근2 컴퓨터 공학2	950002
슈퍼성근3 컴퓨터 공학3	950003
슈퍼성근4 컴퓨터 공학4	950004
슈퍼성근5 컴퓨터 공학5	950005
슈퍼성근6 컴퓨터 공학6	950006
슈퍼성근7 컴퓨터 공학7	950007

■ 얼마나 많이 getView 함수가
호출되겠는가!

■ 따라서 어댑터의 getView 함수는 반드시 최적화되어야 한다.

BaseAdapter를 이용한 리스트뷰 - 어댑터의 최적화

- getView 함수는 총 세 가지 최적화가 이뤄져야 한다.

res/layout/list_view_item_layout.xml

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="10dp">
```

```
<TextView android:id="@+id/name_text"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textSize="20dp"/>
```

```
<TextView android:id="@+id/number_text"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_toRightOf="@id/name_text"
```

```
    android:layout_alignParentRight="true"
```

```
    android:textSize="20dp"
    android:gravity="right"/>
```

```
<TextView android:id="@+id/department_text"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
```

```
    android:layout_below="@id/name_text"/>
```

```
</RelativeLayout>
```

슈퍼성근0

컴퓨터 공학0

950000

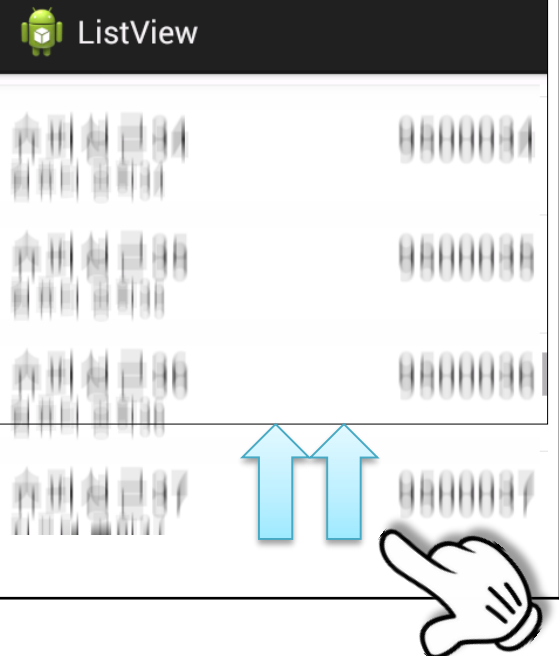
BaseAdapter를 이용한 리스트뷰 - 어댑터의 최적화

② 뷰 재사용하기

매번 getView 함수가 호출될 때마다 inflate 함수를 통해 뷰를 생성한다.

src/BaseAdapterEx.java

```
public class BaseAdapterEx extends BaseAdapter
{
    ...
    @Override
    public View getView( int position,
                        View convertView,
                        ViewGroup parent )
    {
        // 리스트의 한 항목에 해당하는 레이아웃을 생성한다.
        View itemLayout = mLayoutInflater.inflate( R.layout
    }
}
```



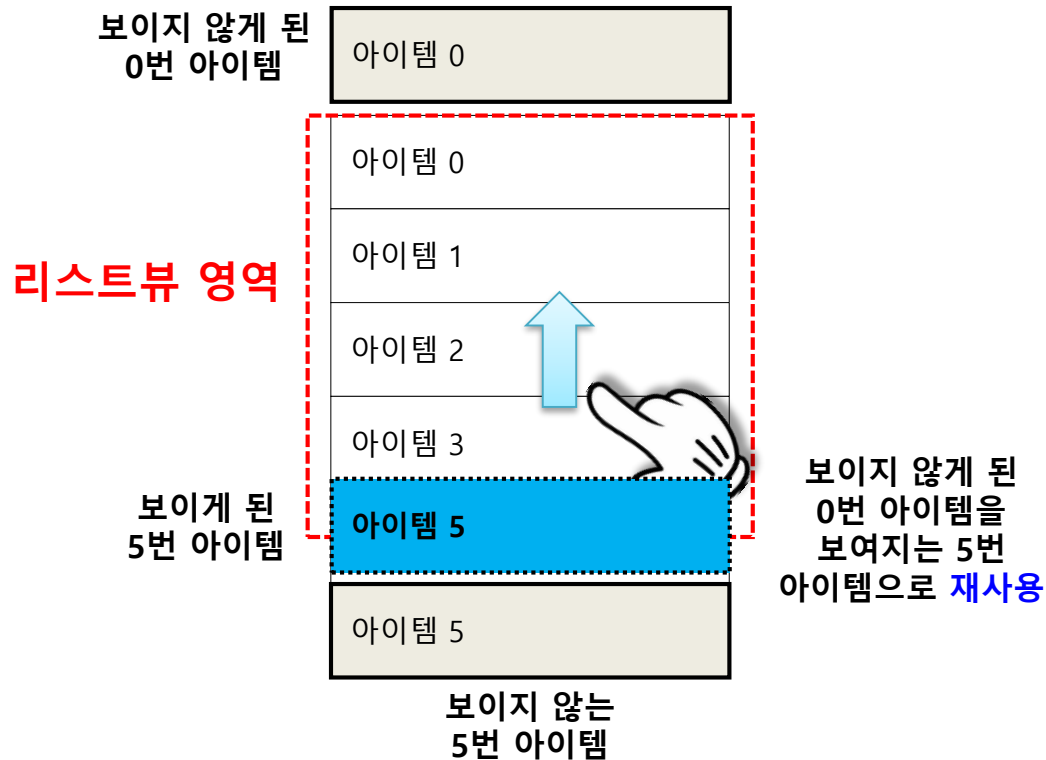
그렇다면 만일 사용자가 리스트뷰를 빠르게 아래위로 플리킹한다면 얼마나 많은 뷰들이 순간적으로 생성될까?

이렇게 비약적으로 늘기만 한다면 앱은 곧 메모리 부족으로 강제 종료되어 버릴 것이다.

직접 확인해보자.

BaseAdapter를 이용한 리스트뷰 - 어댑터의 최적화

이 문제를 해결하기 위해 어댑터뷰는 한 번 생성된 자식뷰를 **재사용**할 수 있도록 지원한다.



따라서 리스트뷰는 화면에 보이는 개수 만큼의 아이템을 가지게 된다.
즉 아이템 개수가 아무리 많아도 전혀 문제되지 않는 것이다.

BaseAdapter를 이용한 리스트뷰 - 어댑터의 최적화

getView 함수 내부를 변경해보자.

src/BaseAdapterEx.java

```
public class BaseAdapterEx extends BaseAdapter
{
    ...
    @Override
    public View getView( int position, View convertView, ViewGroup parent )
    {
        // 1. 어댑터뷰가 재사용할 뷰를 넘겨주지 않은 경우에만 새로운 뷰를 생성한다.
        View itemLayout = convertView;
        if( itemLayout == null )
        {
            itemLayout = mLayoutInflater.inflate(
                R.layout.list_view_item_layout, null);
        }

        // 2. 현재 아이템에 내용을 변경할 뷰를 찾는다.
        TextView nameTv = (TextView)itemLayout.findViewById( R.id.name_text );
        ...

        // 3. 이름, 학번, 학과 데이터를 참조하여 레이아웃을 갱신한다.
        nameTv.setText( mData.get( position ).mName );
        ...

        return itemLayout;
    }
}
```

BaseAdapter를 이용한 리스트뷰 - 어댑터의 최적화

③ View Holer 사용하기

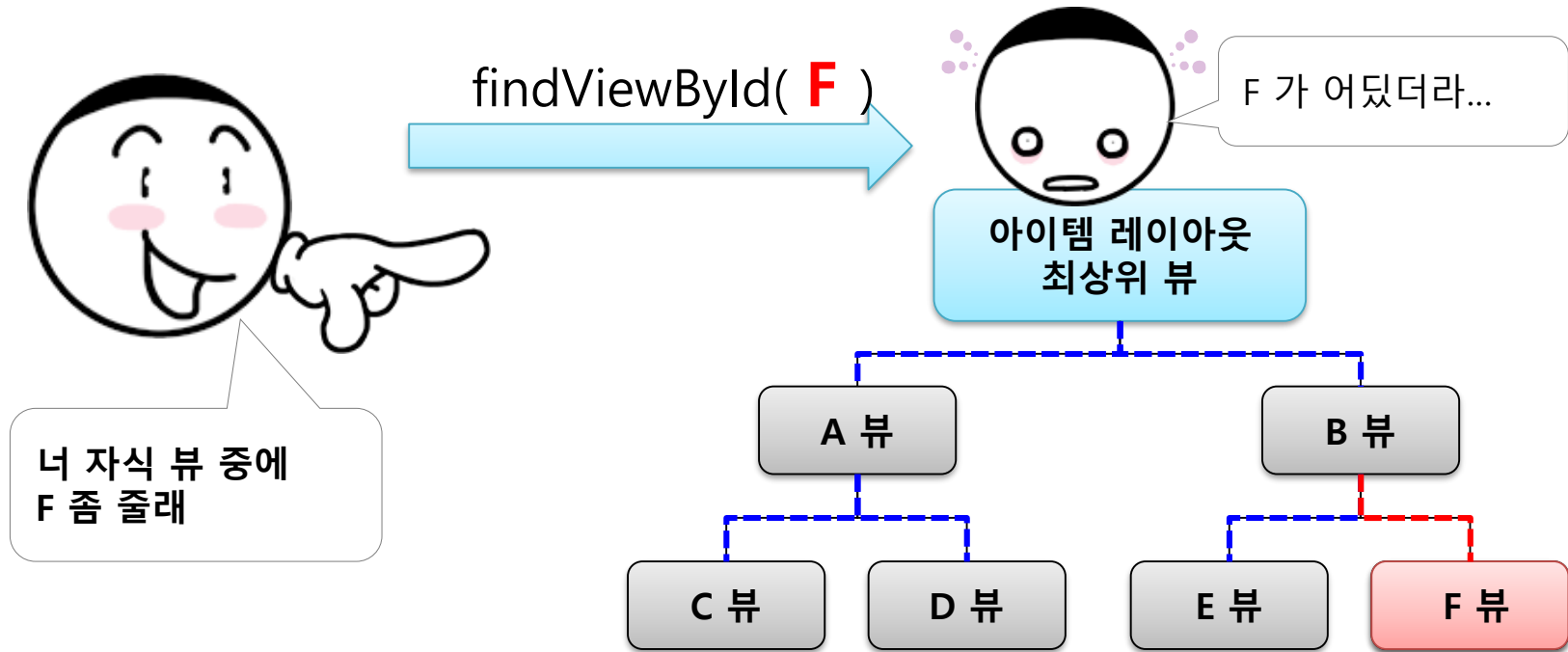
지금까지 수정된 어댑터의 `getView` 함수를 다시 한 번 살펴보자.

`src/BaseAdapterEx.java`

```
public class BaseAdapterEx extends BaseAdapter
{
    ...
    @Override
    public View getView( int position,
                        View convertView,
                        ViewGroup parent )
    {
        ...
        // 현재 아이템에 내용을 변경할 뷰를 찾는다.
        TextView nameTv = (TextView)itemLayout.findViewById( R.id.name_text );
        TextView numberTv =
            (TextView)itemLayout.findViewById( R.id.number_text );
        TextView departmentTv =
            (TextView)itemLayout.findViewById( R.id.department_text );
        ...
    }
}
```

매번 리스트의 한 아이템 레이아웃에서 뷰들을 찾고 있다. 이 것 조차 레이아웃이 복잡할 수록 성능의 영향을 준다.

BaseAdapter를 이용한 리스트뷰 - 어댑터의 최적화



결국 F 뷰는 총 6번의 비교로 찾을 수 있었지만 자식 뷰들이 더 많다면 그만큼 비교 회수가 는다. 이러한 뷰 탐색은 반복문과 비교문을 통해 이뤄지고 성능에 영향을 줄 수 밖에 없다.

■ BaseAdapter를 이용한 리스트뷰 - 어댑터의 최적화

해결 방법으로는 최대한 findViewById 함수 사용을 줄이고,
이를 위해 뷰홀더ViewHolder를 사용하는 것이다.

뷰홀더는 두 번째 최적화 과정이었던 뷰 재사용과 연관이 있는데 **한 번 찾았던 뷰를 더 이상 탐색하지 않도록 저장해두는 방식**을 사용한다.

BaseAdapter를 이용한 리스트뷰 - 어댑터의 최적화

src/BaseAdapterEx.java

```
public class BaseAdapterEx extends BaseAdapter
{
    ...
    class ViewHolder
    {
        TextView mNameTv;
        TextView mNumberTv;
        TextView mDepartmentTv;
    }
}
```

BaseAdapter를 이용한 리스트뷰 - 어댑터의 최적화

```
@Override
public View getView( int position, View convertView, ViewGroup parent )
{
    View      itemLayout = convertView;
    ViewHolder viewHolder = null;

    // 1. 어댑터뷰가 재사용할 뷰를 넘겨주지 않은 경우에만 새로운 뷰를 생성한다.
    if( itemLayout == null )
    {
        itemLayout = mLayoutInflater.inflate( R.layout.list_view_item_layout, null );

        viewHolder = new ViewHolder();
        viewHolder.mNameTv = (TextView) itemLayout.findViewById( R.id.name_text );
        viewHolder.mNumberTv = (TextView) itemLayout.findViewById( R.id.number_text );
        viewHolder.mDepartmentTv = (TextView) itemLayout.findViewById( R.id.department_text );

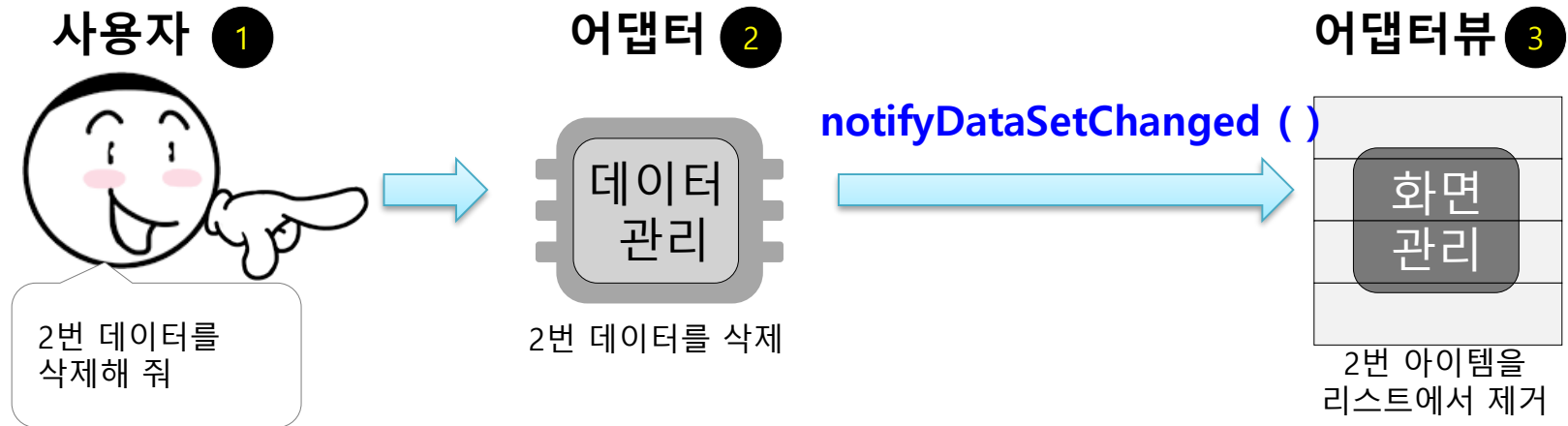
        itemLayout.setTag( viewHolder );
    }
    else
    {
        viewHolder = (ViewHolder) itemLayout.getTag();
    }

    viewHolder.mNameTv.setText( mData.get(position).mName );
    viewHolder.mNumberTv.setText( mData.get(position).mNumber );
    viewHolder.mDepartmentTv.setText( mData.get(position).mDepartment );

    return itemLayout;
}
...
```


BaseAdapter를 이용한 리스트뷰 - 리스트 갱신

- 리스트에서 사용되는 데이터가 변경되면 리스트뷰를 갱신해야 한다.
- 리스트 갱신은 어댑터뷰와 어댑터의 명확한 역할 분담으로 이뤄진다.



- 어댑터는 실제 데이터를 관리한다. 또한 데이터가 갱신되면 반영후 어댑터뷰에게 화면 갱신 요청을 한다.
- 어댑터뷰(리스트뷰)는 화면을 관리한다. 따라서 실제 데이터는 전혀 관리하지 않고 리스트를 그리는 역할만 한다.
- 따라서 어댑터와 어댑터뷰를 구현할 때는 이 역할 분담을 깨선 안 된다.

BaseAdapter를 이용한 리스트뷰 - 리스트 갱신

■ 리스트의 데이터를 갱신하는 기능을 추가해보자.

res/layout/activity_main.xml

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <EditText android:id="@+id/name_edit"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:hint="이름"/>

        <EditText android:id="@+id/number_edit"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:hint="학번"/>

        <EditText android:id="@+id/department_edit"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:hint="학과"/>

        <Button android:id="@+id/add_btn"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="0"
            android:text="추가"
            android:onClick="onClick"/>

    </LinearLayout>
```

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <EditText android:id="@+id/del_item_index_edit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:hint="아이템"/>

    <Button android:id="@+id/del_btn"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="0"
        android:text="삭제"
        android:onClick="onClick"/>

</LinearLayout>
```

ListView			
이름	학번	학과	추가
아이템			삭제 전체 삭제
슈퍼성근0			950000
컴퓨터 공학0			
슈퍼성근1			950001
컴퓨터 공학1			

● 레이아웃 구조

- LinearLayout
 - LinearLayout
 - name_edit (EditText)
 - number_edit (EditText)
 - department_edit (EditText)
 - add_btn (Button) - "추가"
 - LinearLayout
 - del_item_index_edit (EditText)
 - del_btn (Button) - "삭제"
 - all_del_btn (Button) - "전체 삭제"
 - list_view (ListView)

BaseAdapter를 이용한 리스트뷰 - 리스트 갱신

src/BaseAdapterEx.java

```
public class BaseAdapterEx extends BaseAdapter
{
```

```
...
    public void add( int index, Student addData )
    {
        mData.add( index, addData );
        notifyDataSetChanged();
    }
}
```

```
    public void delete( int index )
    {
        mData.remove( index );
        notifyDataSetChanged();
    }
}
```

```
    public void clear( )
    {
        mData.clear();
        notifyDataSetChanged();
    }
}
```

ListView			
이름	학번	학과	추가
아이템	삭제		전체 삭제
슈퍼성근0 컴퓨터 공학0	950000		
슈퍼성근1 컴퓨터 공학1	950001		

BaseAdapter를 이용한 리스트뷰 - 리스트 갱신

src/MainActivity.java

```
public class MainActivity extends Activity
{
    public void onClick( View v )
    {
        switch ( v.getId() )
        {
            case R.id.add_btn:
            {
                EditText nameEt =
                    (EditText) findViewById( R.id.name_edit );
                EditText numberEt =
                    (EditText) findViewById( R.id.number_edit );
                EditText departmentEt =
                    (EditText) findViewById( R.id.department_edit );

                Student addData = new Student();

                addData.mName = nameEt.getText().toString();
                addData.mNumber = numberEt.getText().toString();
                addData.mDepartment = departmentEt.getText().toString();

                mAdapter.add( 0 , addData);

                break;
            }
        }
    }
}
```

BaseAdapter를 이용한 리스트뷰 - 리스트 갱신

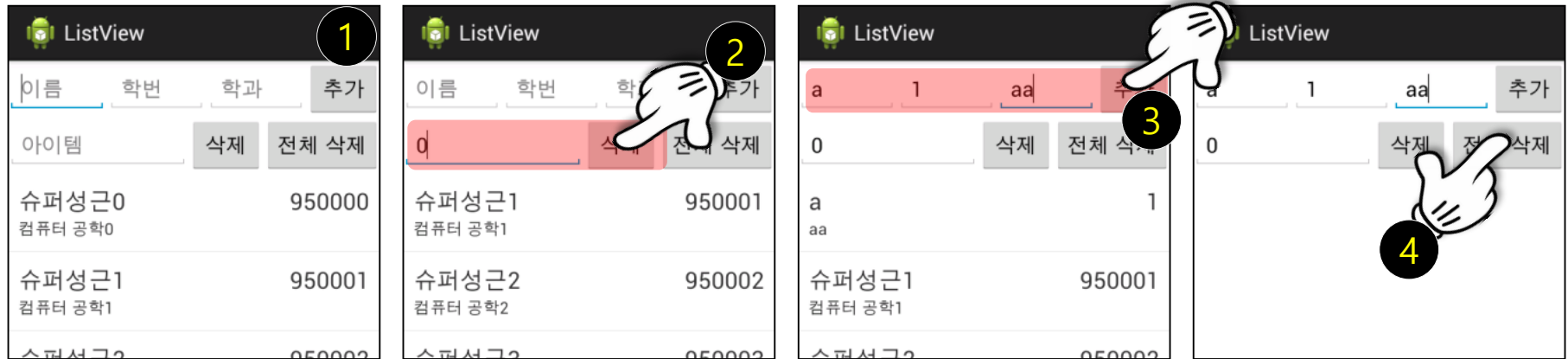
```
case R.id.del_btn:
{
    EditText delItemIndexEt =
        (EditText) findViewById( R.id.del_item_index_edit );

    Integer index =
        Integer.parseInt( delItemIndexEt.getText().toString() );

    mAdapter.delete( index );
    break;
}

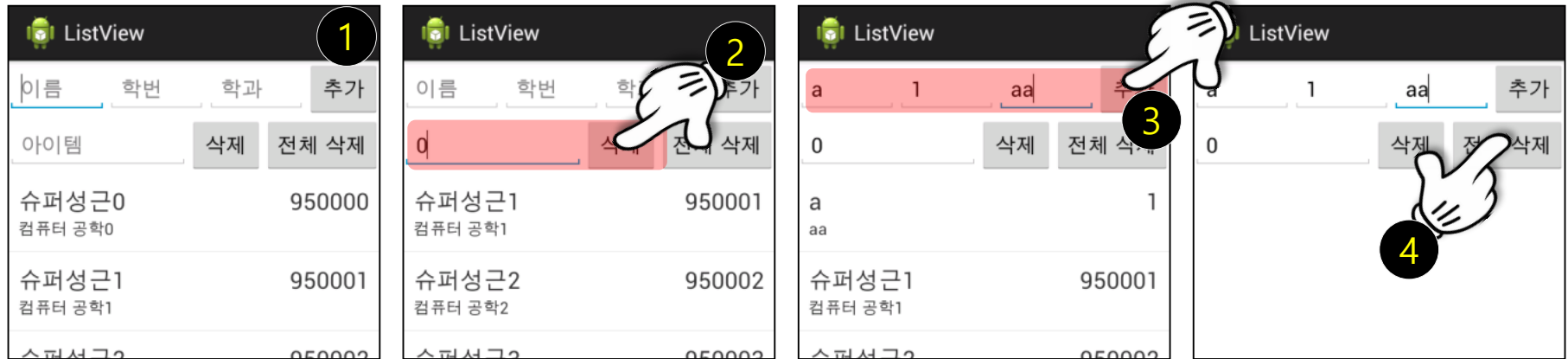
case R.id.all_del_btn:
{
    mAdapter.clear();
    break;
}
}
}
```

BaseAdapter를 이용한 리스트뷰 - 리스트 갱신



- 다시 말하지만 어댑터와 어댑터뷰의 역할 분담은 중요하다.
- 어댑터는 오직 데이터만을 관리하고, 어댑터뷰는 오직 어댑터가 제공하는 데이터만을 기준으로 사용자에게 보여지는 화면을 그린다.
- 그렇다면 예를 들어 리스트 정렬 기능을 추가하고 싶다면 어떻게 해야 할까?
- 정렬은 데이터를 기준으로 한다.
그러므로 데이터를 관리하는 어댑터에 정렬 기능을 추가하고 `notifyDataSetChanged` 함수 호출한다.
- 이후 데이터 갱신을 감지한 어댑터뷰는 알아서 화면을 갱신할 것이다. 하지만 만일 데이터 정렬을 액티비티 등에서 구현한다면 데이터 관리가 분산되어 코드가 복잡해지고 유지보수가 어렵다.

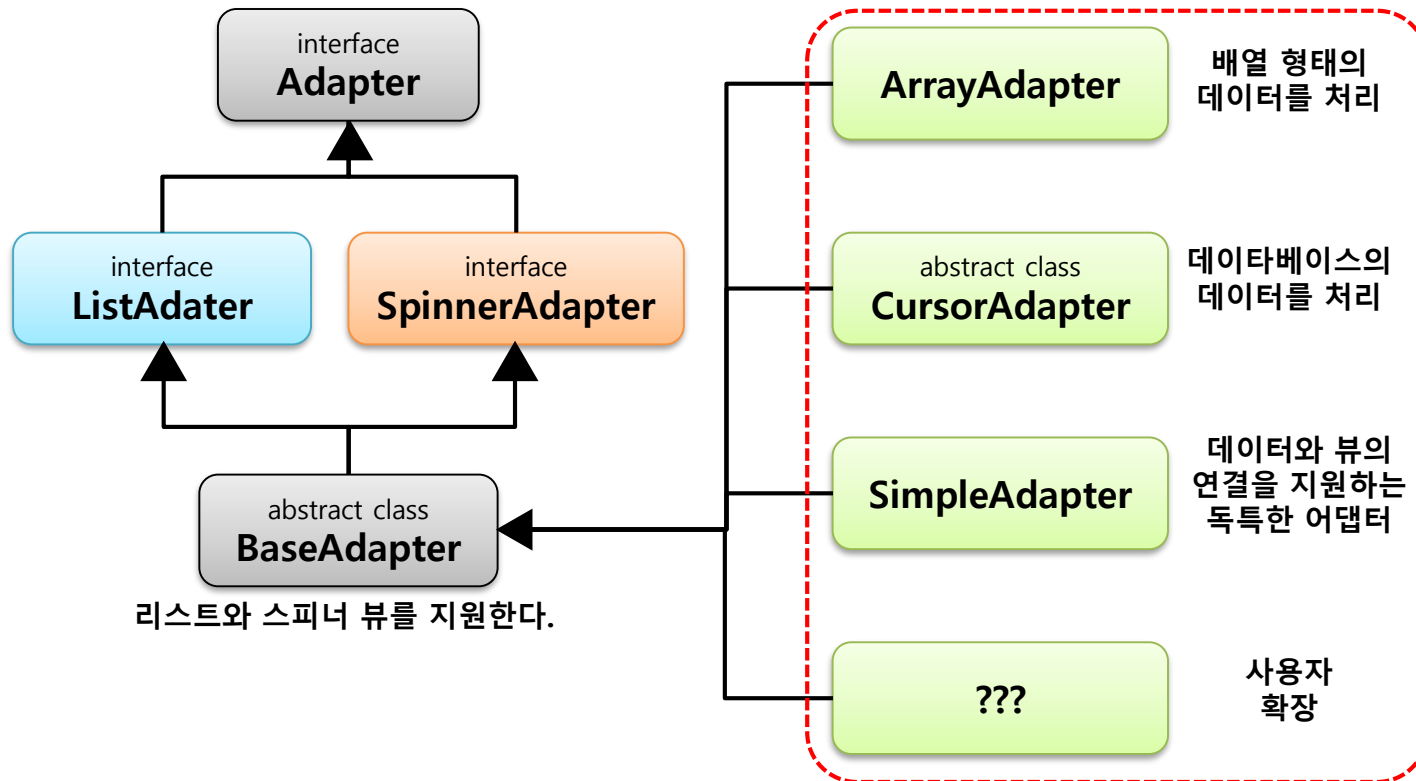
BaseAdapter를 이용한 리스트뷰 - 리스트 갱신



- 다시 말하지만 어댑터와 어댑터뷰의 역할 분담은 중요하다.
- 어댑터는 오직 데이터만을 관리하고, 어댑터뷰는 오직 어댑터가 제공하는 데이터만을 기준으로 사용자에게 보여지는 화면을 그린다.
- 그렇다면 예를 들어 리스트 정렬 기능을 추가하고 싶다면 어떻게 해야 할까?
- 정렬은 데이터를 기준으로 한다.
그러므로 데이터를 관리하는 어댑터에 정렬 기능을 추가하고 `notifyDataSetChanged` 함수 호출한다.
- 이후 데이터 갱신을 감지한 어댑터뷰는 알아서 화면을 갱신할 것이다. 하지만 만일 데이터 정렬을 액티비티 등에서 구현한다면 데이터 관리가 분산되어 코드가 복잡해지고 유지보수가 어렵다.

BaseAdapter에서 파생된 어댑터들

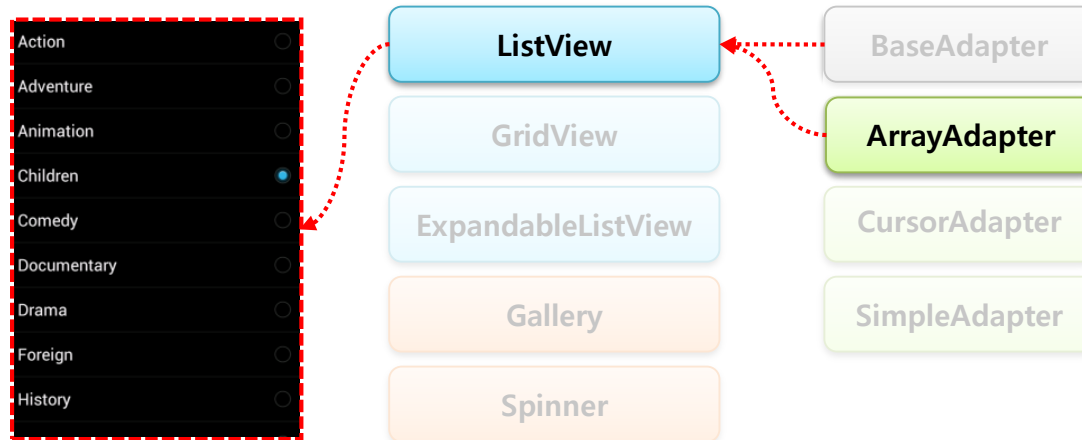
- 지금까지 BaseAdapter를 직접 상속받아 리스트뷰 예제를 구현했다.
- 이를 통해 어댑터의 역할에 대해서 충분히 알게 되었을 것이다.
- 이제부터는 BaseAdapter에서 파생된 클래스들에 대해서 살펴본다.



- BaseAdapter에서 파생된 클래스라면 분명 BaseAdapter에서 일일이 구현해주었던 것들은 뭔가 편하게 기능이 추가된 것이라 유추할 수 있다.

BaseAdapter에서 파생된 어댑터들 - ArrayAdapter

- 지금까지 예제 어댑터에서 사용된 데이터는 리스트 형태 ArrayList였다.
- 하지만 배열 형태의 데이터는 BaseAdapter를 사용하는 것보다 파생된 ArrayAdapter를 사용하는 것이 편하다.
- BaseAdapter를 ArrayAdapter로 변경해보자.



BaseAdapter에서 파생된 어댑터들 - ArrayAdapter

1.. 라이브러리의 Adapter

ArrayAdapter

- 항목에 문자열 데이터를 순서대로 하나씩 나열

```
<ListView  
    android:id="@+id/main_list"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
>
```

```
String[] datas=getResources().getStringArray(R.array.location);  
ArrayAdapter<String> adapter=new ArrayAdapter<String>(this,  
    android.R.layout.simple_list_item_1, datas);  
listView.setAdapter(adapter);
```

- this: Context 객체
- android.R.layout.simple_list_item1: 항목 하나를 구성하기 위한 레이아웃 XML 파일 정보
- datas: 항목을 구성하는 데이터

ListView를 위해 제공되는 라이브러리의 XML 파일

- simple_list_item_1: 항목에 문자열 데이터 하나
- simple_list_item_2: 항목에 문자열 데이터 두 개 위아래 나열
- simple_list_item_multiple_choice: 문자열과 오른쪽 체크박스 제공
- simple_list_item_single_choice: 문자열과 오른쪽 라디오버튼 제공



BaseAdapter에서 파생된 어댑터들 - ArrayAdapter

- 개발자가 항목 레이아웃 XML을 직접 만들어 적용

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">
    <ImageView
        android:id="@+id/main_item_icon"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:src="@drawable/icon"
        android:maxLength="20dp"
        android:maxLength="20dp"
        android:adjustViewBounds="true"/>
    <TextView
        android:id="@+id/main_item_name"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_toRightOf="@id/main_item_icon"
        android:layout_marginLeft="16dp"/>
    <CheckBox
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentRight="true"/>
</RelativeLayout>
```



BaseAdapter에서 파생된 어댑터들 - ArrayAdapter

- ArrayAdapter에게 데이터를 출력할 TextView가 어떤 뷰인지 id값 지정

```
String[] datas=getResources().getStringArray(R.array.location);
ArrayAdapter<String> adapter=new ArrayAdapter<String>(this,
    R.layout.main_item, R.id.main_item_name, datas);
listView.setAdapter(adapter);
```

- 항목 선택 이벤트 처리

```
public class MainActivity extends AppCompatActivity implements
AdapterView.OnItemClickListener{
    String[] datas;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        //..... listView.setOnItemClickListener(this);
    }

    @Override
    public void onItemClick(AdapterView<?> parent, View view, int position, long id) {
        Toast t=Toast.makeText(this, datas[position], Toast.LENGTH_SHORT);
        t.show();
    }
}
```

BaseAdapter에서 파생된 어댑터들 - ArrayAdapter

src/ArrayAdapterEx.java

```
public class ArrayAdapterEx extends ArrayAdapter<Student>
{
    public ArrayAdapterEx (Context context, int resource, int textViewResourceId, List<Student> data )
    {
        super( context, resource, textViewResourceId, data );
    }

    class ViewHolder { TextView mNameTv; TextView mNumberTv; TextView mDepartmentTv; }

    @Override
    public View getView( int position, View convertView, ViewGroup parent )
    {
        View itemLayout = super.getView( position, convertView, parent );

        ViewHolder viewHolder = (ViewHolder)itemLayout.getTag();
        if( viewHolder == null ) {
            viewHolder = new ViewHolder();
            viewHolder.mNameTv =
                (TextView) itemLayout.findViewById( R.id.name_text );
            viewHolder.mNumberTv =
                (TextView) itemLayout.findViewById( R.id.number_text );
            viewHolder.mDepartmentTv =
                (TextView) itemLayout.findViewById( R.id.department_text );
            itemLayout.setTag( viewHolder );
        }

        viewHolder.mNameTv.setText( getItem( position ).mName );
        viewHolder.mNumberTv.setText( getItem( position ).mNumber );
        viewHolder.mDepartmentTv.setText( getItem( position ).mDepartment );
        return itemLayout; ...
    }
}
```

BaseAdapter에서 파생된 어댑터들 - ArrayAdapter

src/MainActivity.java

```
public class MainActivity extends Activity
{
    ...
    ArrayAdapterEx mAdapterer = null;

    protected void onCreate( Bundle savedInstanceState )
    {
        ...
        mAdapterer = new ArrayAdapterEx( this,
                                         R.layout.list_view_item_layout,
                                         R.id.name_text,
                                         mData );
        ...
    }
}
```

BaseAdapter에서 파생된 어댑터들 - ArrayAdapter

```
public void onClick( View v )
{
    switch ( v.getId() )
    {
        case R.id.add_btn:
        {
            ...
            mAdapter.add( addData );
            break;
        }

        case R.id.del_btn:
        {
            ...
            mAdapter.remove( mData.get( index ) );
            break;
        }

        case R.id.all_del_btn:
        {
            ...
            mAdapter.clear();
            break;
        }
        ...
    }
}
```

하지만 ArrayAdapter에는 데이터를 갱신하는 기능을 추가한 적이 없다.

ArrayAdapter에는 기본적으로 데이터 갱신 기능이 구현되어 있다.

참 편리하지 않은가!

이 밖에도 ArrayAdapter는 데이터 정렬 기능도 제공된다.

BaseAdapter에서 파생된 어댑터들 - ArrayAdapter

src/MainActivity.java

```
public class MainActivity extends Activity
{
    ...
    public void onClick( View v )
    {
        switch ( v.getId() )
        {
            ...
            case R.id.sort_btn:
            {
                mAdapter.sort(
                    new Comparator<Student>()
                    {
                        @Override
                        public int compare( Student lhs, Student rhs )
                        {
                            Collator collator = Collator.getInstance();
                            return collator.compare( rhs.mName, lhs.mName );
                        }
                    }
                );
                break;
            }
            ...
        }
    }
}
```

ListView	
이름	학번
아이템	삭제
슈퍼성근0	950000
컴퓨터 공학0	
슈퍼성근1	950001
컴퓨터 공학1	
슈퍼성근2	950002



- 첫 번째 매개변수 > 두 번째 매개변수 : 양수 반환(교환)
- 두 수가 모두 동일한 경우 : 0 값 반환
- 첫 번째 매개변수 < 두 번째 매개변수 : 음수 반환

BaseAdapter에서 파생된 어댑터들 - ArrayAdapter

ListView			
이름	학번	학과	추가
아이템	삭제	전체 삭제	정렬
슈퍼성근0 컴퓨터 공학0			950000
슈퍼성근1 컴퓨터 공학1			950001
슈퍼성근2 컴퓨터 공학2			950002



ListView			
이름	학번	학과	추가
아이템	삭제	전체 삭제	정렬
슈퍼성근99 컴퓨터 공학99			9500099
슈퍼성근98 컴퓨터 공학98			9500098
슈퍼성근97 컴퓨터 공학97			9500097



BaseAdapter에서 파생된 어댑터들 - SimpleAdapter

■ SimpleAdapter는 의미 그대로 매우 단순한 어댑터다.

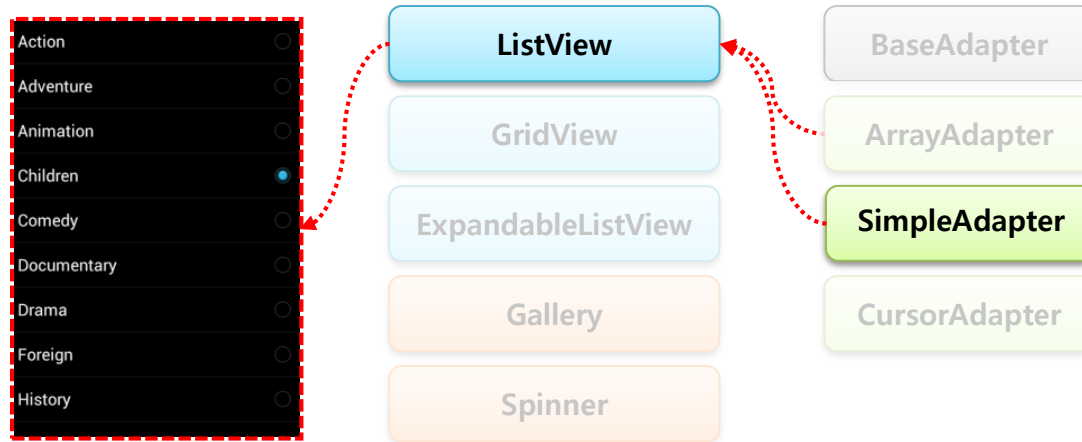
■ 또한 단순하다는 것은 쓰임새가 매우 제한적이라는 의미도 담고 있다. 어떤 제한을 가지는지 알아보자.

- 내부 데이터는 ArrayList를 사용하나 리스트의 값은 맵 형식이어야 한다.
즉 예제와 같이 ArrayList값으로 Student 객체를 사용할 수 없고,
키와 값으로 이뤄진 맵 형식이어야 한다.
- 맵의 데이터와 그 데이터가 보여질 뷰를 연결해야 한다.



BaseAdapter에서 파생된 어댑터들 - SimpleAdapter

■ ArrayAdapter를 SimpleAdapter로 변경해보자.



BaseAdapter에서 파생된 어댑터들 - SimpleAdapter

SimpleAdapter의 생성자

- this: Context 객체
- datas: 항목 구성을 위한 데이터. ArrayList<HashMap<String,String>>
- android.R.layout.simple_list_item_2: 한 항목을 위한 레이아웃 XML
- new String[]{"name","content"}: 한 항목의 데이터를 가지는 HashMap에서

데이터를 추출하기 위한 키 값

- new int[]{android.R.id.text1, android.R.id.text2}: 추출된 데이터를 화면에 출력하기 위한 레이아웃 파일 내의 뷰 id 값

BaseAdapter에서 파생된 어댑터들 - SimpleAdapter

src/MainActivity.java

```
public class MainActivity extends Activity
{
    ListView                                mListView = null;

    ArrayList<HashMap<String, String>>    mData      = null;
    SimpleAdapter                          mAdapter    = null;

    @Override
    protected void onCreate( Bundle savedInstanceState )
    {
        super.onCreate( savedInstanceState );
        setContentView( R.layout.activity_main );

        // 1. 뷰와 그 뷰에 설정될 데이터를 연결하기 위한 배열을 정의한다.
        String[] dataKeyList = { "name",           "number",           "department" };

        int[]   viewIdList   = { R.id.name_text,  R.id.number_text,  R.id.department_text };

        // 2. 어댑터에서 사용할 데이터 설정
        mData = new ArrayList<HashMap<String, String>>();
    }
}
```

BaseAdapter에서 파생된 어댑터들 - SimpleAdapter

```
for( int i = 0 ; i < 100 ; i++ )  
{  
    HashMap<String, String> mapData = new HashMap<String, String>();  
    mapData.put( dataKeyList[0], "슈퍼성근" + i);  
    mapData.put( dataKeyList[1], "95000" + i);  
    mapData.put( dataKeyList[2], "컴퓨터 공학" + i);  
    mData.add( mapData );  
}
```

// 3. 어댑터를 생성하고 데이터 설정

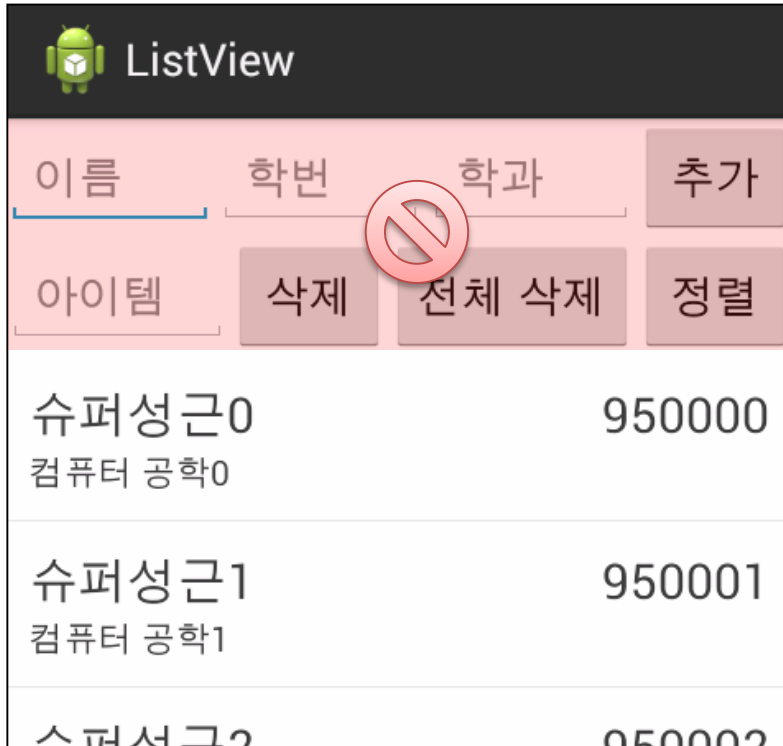
```
mAdapter = new SimpleAdapter( this,  
                             mData,  
                             R.layout.list_view_item_layout,  
                             dataKeyList,  
                             viewIdList );
```

```
String[] dataKeyList = { "name", "number", "department" };  
int[] viewIdList = { R.id.name_text, R.id.number_text, R.id.department_text };
```

BaseAdapter에서 파생된 어댑터들 - SimpleAdapter

■ SimpleAdapter는 별도로 상속받아 구현하지 않아도 된다.

■ 뷰와 뷰에 그려질 맵 데이터만 연결해 주면 되므로 getView 함수를 구현할 필요가 없기 때문이다.



■ 하지만 SimpleAdapter는 데이터 갱신이나 정렬 기능은 제공하지 않는다.

■ 간단히 리스트를 보여주는 용도일뿐이다.

■ 그러므로 복잡한 리스트 아이템 레이아웃을 구성할 수도 없고, 데이터 형식은 꼭 맵을 써야 하는 단점이 있다.

■ 물론 SimpleAdapter를 상속받아 getView를 직접 구현하면 복잡한 아이템 레이아웃을 구성할 수도 있다.

하지만 그렇다면 굳이 SimpleAdapter를 사용하지 않고 ArrayAdapter를 쓰면 되지 않겠는가!